Al al-Bayt University - Prince Al Hussein bin Abdullah II Faculty of Information Technology

Course Information and Database Security (906313)
First Semester 2026/2027 - Final Semester Project

SecARS

Secure Academic Records System

Requirements Document

Functional requirements FR01-FR12, business rules and use cases for the four roles

Group members
Abdullah Ibrahim Hassan
Abdulrahman Awartani
[Member 3 - name to be added]

Milestone M1 - Analysis & Design | Due Friday 06/11/2026
Version 1.0 | November 2026

1. Introduction

SecARS (Secure Academic Records System) is a university application that manages students, lecturers, courses, course sections, enrolments and grades on top of an Oracle database. The point of the project is not the application features but the security around them: identity and privileges, protected queries, auditing of grade changes, encryption, backup/restore and masking, each proved by repeatable tests.

This document fixes what the system must do (the 12 functional requirements FR01-FR12 from the course specification, refined for SecARS), the business rules that the database and the application must enforce, and the use cases of the four roles: Student, Lecturer, Registrar and Security Auditor. It is the baseline for the ERD, the architecture/DFD, the STRIDE model and all later milestones.

1.1 Technology baseline (from the approved specification)

Layer	Choice
Database	Oracle AI Database Free 26ai (service FREEPDB1); schemas ARS_OWNER (objects), APP_USER (service account), ARS_LAB (isolated SQL-injection lab)
Backend	Java 21, Spring Boot 3.x, Spring Security, Maven, Oracle JDBC; PL/SQL package ARS_API as the only data interface
Frontend	React + Vite + Node.js LTS; four role screens
Tools	SQL Developer/SQLcl, JUnit/MockMvc, Postman, sqlmap (LAB only), Git (private GitHub repository)

1.2 Assumptions and constraints

- All data is fictional and generated by the group's own seed script (minimum 200 students, 15 lecturers, 40 courses, 40 sections, 400 enrolments, plus test accounts for all four roles; at least two registrars so the four-eyes rule can be tested).
- Oracle Free limits (2 CPUs, 2 GB RAM, 12 GB user data) apply. VPD, FGA, Unified Auditing, TDE and TCPS availability will be verified on the installed edition at the start of M2; any limitation is reported to the instructor before M3.
- Attacks and scans are performed only against the LAB schema on localhost.
- Out of scope: payments, timetables, e-mail/SMS notifications, password-reset by e-mail, real student data, production deployment.

2. Roles and responsibilities

Role	Who	May do	Must never do
Student	Enrolled student	View own profile, own enrolments and PUBLISHED grades	See another student's data, change grades, see audit records
Lecturer	Teaching staff	See own sections and their students; enter grades while the section is OPEN; submit the section	Change grades in other sections; edit after SUBMITTED without the workflow; approve changes
Registrar	Registry office	Manage students, courses, sections, enrolments; publish sections; request and approve (a different registrar) post-closure grade changes	Delete or edit audit records; approve own request
Security Auditor	Security staff	Read GRADE_AUDIT, SECURITY_EVENTS and reports	Read or modify academic data; change any grade

2.1 Grade lifecycle

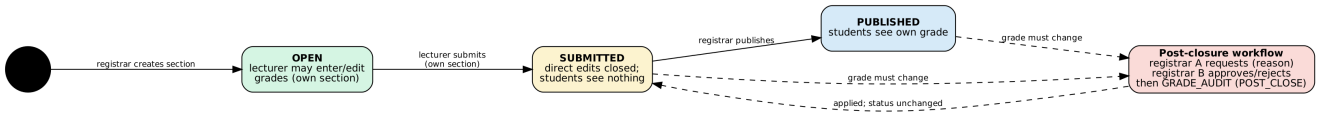


Figure 1 - Section status drives who may touch a grade. After SUBMITTED, changes only through the registrar workflow with a reason.

3. Functional requirements FR01-FR12

FR01 Secure login, logout and session expiry	Role: All	Milestone: M2
Description	A user signs in with username and password. The backend verifies the BCrypt/Argon2 hash in APP_USERS, refuses disabled or locked accounts, and creates a server-side session (HttpOnly, SameSite cookie, new session ID on login). Logout invalidates the session. Idle sessions expire after 15 minutes.	
Acceptance criteria	Valid credentials open a session and return the role from the server; wrong password, disabled account and locked account are all refused with the same generic message; after 5 consecutive failures the account is locked for 15 minutes and a LOGIN_FAILED / ACCOUNT_LOCKED row is written to SECURITY_EVENTS; after logout or expiry any API call returns 401.	
Interface	POST /api/auth/login, POST /api/auth/logout, GET /api/me	
Related security requirements	SR01, SR02	

FR02 Student self-service screen	Role: Student	Milestone: M2 / M3
Description	A student sees only their own profile, their enrolments and the grades of sections whose status is PUBLISHED. Grades of OPEN or SUBMITTED sections are shown as 'not published'.	
Acceptance criteria	T1 passes (own published grades only); changing the student ID in the request returns 403/404 and no foreign rows (T2); no grade is returned for non-PUBLISHED sections.	
Interface	GET /api/students/me/profile, /enrollments, /grades	
Related security requirements	SR02, SR04	

FR03 Lecturer screen for own sections	Role: Lecturer	Milestone: M2 / M3
Description	A lecturer sees the list of the sections they teach and the students enrolled in each. Sections taught by other lecturers are not listed and cannot be opened by ID.	
Acceptance criteria	Only sections whose lecturer_id equals the logged-in lecturer are returned; requesting another section ID is refused and logged (T4 access-attempt log).	
Interface	GET /api/lecturer/offerings, /offerings/{id}/students	
Related security requirements	SR02, SR04	

FR04 Grade entry within section status	Role: Lecturer	Milestone: M2 / M3
Description	A lecturer enters or edits grades (0-100) for students of their own section while its status is OPEN, then submits the section (status SUBMITTED), which closes direct editing.	
Acceptance criteria	Entry succeeds only for own section in OPEN state; entry in SUBMITTED/PUBLISHED or in another lecturer's section is refused; every accepted change creates a GRADE_AUDIT row (ENTRY/EDIT) with the real actor.	
Interface	POST /api/lecturer/offerings/{id}/grades, POST /api/lecturer/offerings/{id}/submit	
Related security requirements	SR02, SR07, SR08	

FR05 Registrar record management	Role: Registrar	Milestone: M2
Description	The registrar creates and edits students, courses, sections (including assigning the lecturer) and enrolments, and publishes a SUBMITTED section so students can see grades. The registrar cannot touch audit tables.	
Acceptance criteria	CRUD operations succeed for registrar only; unique and CHECK constraints reject duplicates and invalid values (duplicate enrolment, credit hours out of range); delete of a record that has dependants is refused.	
Interface	/api/registrar/students, /courses, /offerings, /enrollments, POST /api/registrar/offerings/{id}/publish	
Related security requirements	SR02, SR03, SR08	

FR06 Post-closure grade-change workflow	Role: Registrar	Milestone: M2 / M3
Description	After a section is SUBMITTED, a grade can change only through a request: registrar A files a request with a mandatory reason; a different registrar B approves or rejects it. Approval applies the new grade and writes the audit record in one transaction.	
Acceptance criteria	A request without a reason is rejected; the requester cannot approve their own request (ck_sod); a lecturer cannot approve; after approval GRADE_AUDIT shows actor, old/new value, time, target record and reason (T9).	
Interface	POST /api/registrar/grade-changes, POST /api/registrar/grade-changes/{id}/approve | /reject	
Related security requirements	SR07, SR08	

FR07 Auditor read-only views	Role: Security Auditor	Milestone: M2 / M3
Description	The auditor reads the grade-change log (GRADE_AUDIT), security events (failed logins, denied access) and summary reports, with filtering by date, actor and event type. The auditor has no write or edit privilege anywhere.	
Acceptance criteria	Auditor sees audit/security data (T-view); any attempt to read academic data or update a grade is refused (T6); registrar/lecturer/student requesting audit data are refused (T5).	
Interface	GET /api/auditor/events, /grade-audit, /reports/summary	
Related security requirements	SR02, SR08	

FR08 Role-appropriate search, sort and filter	Role: All (per role)	Milestone: M2 / M3
Description	Each role can search, sort and filter only the record sets it is allowed to see (e.g. registrar searches students by name; lecturer filters own roster; auditor filters events by date). Dynamic ORDER BY uses an allow-list; all inputs are validated and bound.	
Acceptance criteria	Injection payloads in search and sort parameters return no extra data and no database error (T7/T8 on the secured app); unknown sort column is rejected; page size is capped.	
Interface	query parameters on the GET endpoints above	
Related security requirements	SR05, SR06	

FR09 Non-revealing error messages	Role: All	Milestone: M2 / M3
Description	Errors returned to the client contain a generic message and a correlation ID only. SQL text, ORA- codes, stack traces, table names and credentials appear only in the server log.	
Acceptance criteria	Forcing a constraint violation, malformed JSON and a DB outage never exposes ORA-/SQL/stack-trace text in any HTTP response.	
Interface	global exception handler (all endpoints)	
Related security requirements	SR06	

FR10 Run from scratch from README and SQL files	Role: Project team / instructor	Milestone: M2 / M4
Description	A clean machine can rebuild the whole system by following README.md and running the numbered SQL scripts in order, then starting backend and frontend with environment variables from .env.example.	
Acceptance criteria	Instructor (or a teammate on a clean machine) reaches a working login of all four roles with no undocumented step; lab-only reset script leaves other databases untouched.	
Interface	db/01_admin_setup.sql ... db/security/*.sql, README.md	
Related security requirements	SR10, SR14	

FR11 Simple, usable interfaces	Role: All	Milestone: M2
Description	Four dedicated, plain React screens (student, lecturer, registrar, auditor) with navigation, forms and tables. Visual polish is not a goal; hiding a button is never an authorisation mechanism.	
Acceptance criteria	Each role reaches only its own screen after login; forcing a URL or API call of another role is refused by the backend.	
Interface	React routes /student, /lecturer, /registrar, /auditor	
Related security requirements	SR02	

FR12 Vulnerable lab separated from the real database	Role: Project team	Milestone: M3
Description	The deliberately injectable search function lives in a separate ARS_LAB schema and a separate lab/ application bound to localhost only, with fictional data and its own account. It never connects to the ARS_OWNER schema.	
Acceptance criteria	The LAB account has no privilege on ARS_OWNER objects and vice versa; lab/README_SAFETY.md states the rules; T7/T8 run only against the LAB.	
Interface	lab/isolated_SQLi_demo/, schema ARS_LAB	
Related security requirements	SR05, SR13	

4. Business rules

The rules below are enforced twice where possible: by the database (constraints, ARS_API, VPD) and by the backend. A rule enforced only in the React interface does not count.

ID	Rule	Statement	Enforced by
BR01	Grade range	A grade is a number from 0 to 100 with up to two decimals; NULL means not yet entered.	CHECK on ENROLLMENTS.grade
BR02	Grade-entry lifecycle	Section status moves only OPEN -> SUBMITTED -> PUBLISHED. The lecturer who teaches the section performs OPEN -> SUBMITTED; the registrar performs SUBMITTED -> PUBLISHED. No backward transition except through the post-closure workflow.	ARS_API package + CHECK on status
BR03	Direct edits	Direct grade edits by the lecturer are allowed only while the section is OPEN and only in a section where COURSE_OFFERINGS.lecturer_id equals the lecturer's own ID.	ARS_API + VPD
BR04	Post-closure change	Once a section is SUBMITTED or PUBLISHED, a grade changes only via an approved GRADE_CHANGE_REQUESTS record with a reason of at least 10 characters.	ARS_API + CHECK
BR05	Four-eyes approval	The registrar who files a grade-change request cannot approve or reject it; a different registrar must decide. Lecturers and students can never approve.	ck_sod + ARS_API
BR06	Grade visibility	A student sees a grade only when the section status is PUBLISHED and the enrolment belongs to that student.	VPD on ENROLLMENTS + API
BR07	Unique enrolment	A student can be enrolled at most once in a given section.	UNIQUE (student_id, offering_id)
BR08	Unique section	The same course cannot have two sections with the same number in the same term; a section has exactly one lecturer.	UNIQUE (course_id, term, section_no)
BR09	Capacity	The number of enrolments in a section cannot exceed its capacity.	ARS_API check
BR10	Credit hours	A course has between 1 and 4 credit hours and a unique code.	CHECK + UNIQUE
BR11	Account-identity link	A STUDENT account links to exactly one student, a LECTURER account to exactly one lecturer; REGISTRAR and AUDITOR accounts link to none.	ck_user_identity
BR12	Authentication policy	Passwords are stored only as BCrypt/Argon2 hashes; minimum 10 characters; 5 consecutive failures lock the account for 15 minutes; disabled accounts (active = 0) can never log in.	Spring Security + APP_USERS
BR13	Session policy	Server-side session, 15-minute idle timeout, new session ID after login, cookie HttpOnly + SameSite, CSRF token on state-changing requests.	Spring Security
BR14	Audit immutability	GRADE_AUDIT and SECURITY_EVENTS are append-only: no application role (including APP_USER) can UPDATE or DELETE them; only the auditor can read them.	GRANTs + SR08
BR15	No deletion of referenced data	Records referenced by other records (students with enrolments, sections with enrolments, etc.) cannot be deleted; no ON DELETE CASCADE is used.	FK NO ACTION
BR16	Identity from the server only	The role and user ID used for authorisation always come from the server session, never from a request parameter, header or body sent by the client.	Spring Security + ARS_CTX
BR17	Fictional data only	All data is generated by the group's own seed script; no real student, lecturer or national-ID data is used anywhere, including screenshots.	db/03_seed.sql

5. Use cases

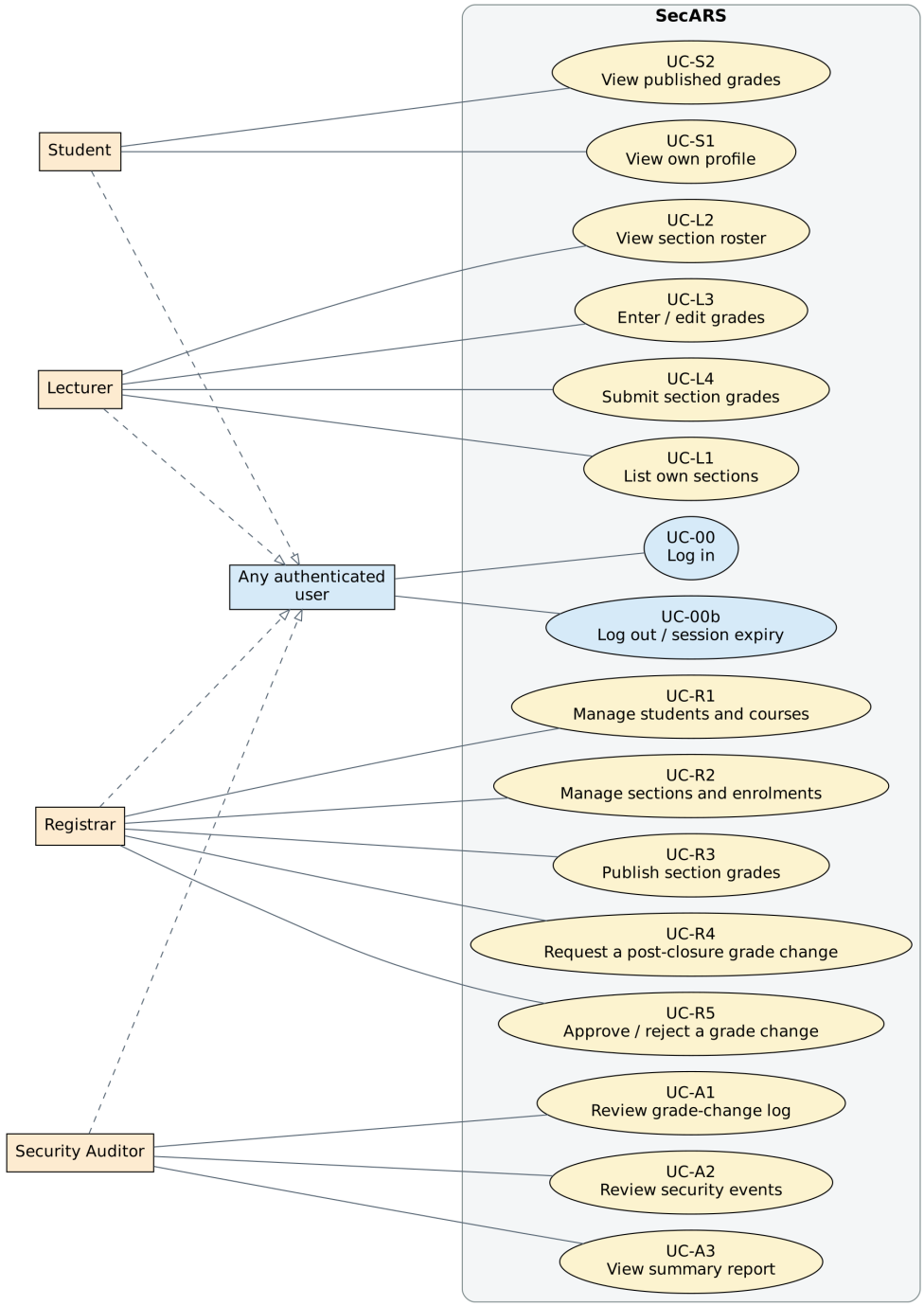


Figure 5 - Use cases per role (blue = common use cases available to every authenticated role). Each use case below includes its exception flows because the refused cases are the security-relevant ones.

5.1 Common use cases

UC-00 Log in	Actor: Any user	FR: FR01
Precondition	Account exists and is active; not locked.	
Main flow	1. User opens the login page and enters username and password. 2. Backend looks up the user and verifies the password hash. 3. Backend creates a session and returns the role. 4. React navigates to the screen for that role.	
Alternative / exception flows	- Wrong password: generic 'invalid credentials', failed counter +1, LOGIN_FAILED event. - 5th consecutive failure: ACCOUNT_LOCKED for 15 minutes. - active = 0: refused with the same generic message.	
Postcondition	Authenticated session exists; SECURITY_EVENTS has the outcome.	

UC-00b Log out / session expiry	Actor: Any user	FR: FR01
Precondition	Authenticated session.	
Main flow	1. User presses Logout (or idle timeout is reached). 2. Backend invalidates the session and clears the cookie.	
Alternative / exception flows	- Any later API call with the old cookie returns 401.	
Postcondition	No valid session remains.	

5.2 Student

UC-S1 View own profile	Actor: Student	FR: FR02, FR09
Precondition	Logged in as STUDENT.	
Main flow	1. Student opens 'My profile'. 2. Backend takes the student ID from the server session (not from the request). 3. Profile and enrolled sections are returned.	
Alternative / exception flows	- Request carries another student's ID: ignored/refused, IDOR_BLOCKED event.	
Postcondition	Student sees own data only.	

UC-S2 View published grades	Actor: Student	FR: FR02
Precondition	Logged in; at least one enrolment.	
Main flow	1. Student opens 'My grades'. 2. ARS_API returns enrolments of this student whose section status is PUBLISHED, with grade. 3. Other sections are listed as 'not published'.	
Alternative / exception flows	- Student changes studentId/enrollmentId in the request: refused at the backend; VPD would return no rows (T2).	
Postcondition	Grades shown only after PUBLISHED.	

5.3 Lecturer

UC-L1 List own sections	Actor: Lecturer	FR: FR03
Precondition	Logged in as LECTURER.	
Main flow	1. Lecturer opens 'My sections'. 2. Backend returns sections whose lecturer_id is the lecturer's ID.	
Alternative / exception flows	- Opening a section ID owned by another lecturer: 403 and ACCESS_DENIED event.	
Postcondition	Only own sections visible.	

UC-L2 View section roster	Actor: Lecturer	FR: FR03, FR08
Precondition	Section belongs to the lecturer.	
Main flow	1. Lecturer opens a section. 2. System lists enrolled students (name, student ID) and current grades.	
Alternative / exception flows	- Roster of another section: refused and logged. National ID, DOB, phone are never returned.	
Postcondition	Roster displayed.	

UC-L3 Enter / edit grades	Actor: Lecturer	FR: FR04
Precondition	Own section, status OPEN.	
Main flow	1. Lecturer types grades for one or more students. 2. ARS_API validates range, ownership and status. 3. Grade stored; trigger/package writes GRADE_AUDIT (ENTRY or EDIT).	
Alternative / exception flows	- Status SUBMITTED/PUBLISHED: refused ('entry closed'). - Grade outside 0-100: validation error. - Section of another lecturer (T4): refused and logged.	
Postcondition	Grades saved with audit trail.	

UC-L4 Submit section grades	Actor: Lecturer	FR: FR04
Precondition	Own section, status OPEN.	
Main flow	1. Lecturer presses Submit. 2. Status becomes SUBMITTED; direct edits are now closed.	
Alternative / exception flows	- Lecturer tries to set PUBLISHED: refused.	
Postcondition	Section SUBMITTED.	

5.4 Registrar

UC-R1 Manage students and courses	Actor: Registrar	FR: FR05, FR09
Precondition	Logged in as REGISTRAR.	
Main flow	1. Registrar creates or edits a student or course. 2. Backend validates input; database constraints enforce uniqueness and ranges.	
Alternative / exception flows	- Duplicate national ID / course code: constraint error mapped to a generic message. - Delete of a record with dependants: refused.	
Postcondition	Record saved.	

UC-R2 Manage sections and enrolments	Actor: Registrar	FR: FR05
Precondition	Courses and lecturers exist.	
Main flow	1. Registrar creates a section (course, lecturer, term, number, capacity). 2. Registrar enrols a student in a section.	
Alternative / exception flows	- Duplicate enrolment or full section: refused. - Enrolment in a non-OPEN section: refused.	
Postcondition	Section / enrolment saved.	

UC-R3 Publish section grades	Actor: Registrar	FR: FR05
Precondition	Section status SUBMITTED.	
Main flow	1. Registrar presses Publish. 2. Status becomes PUBLISHED; students can now see their grades.	
Alternative / exception flows	- Section still OPEN: refused.	
Postcondition	Grades visible to students.	

UC-R4 Request a post-closure grade change	Actor: Registrar (requester)	FR: FR06
Precondition	Section SUBMITTED or PUBLISHED; enrolment has a grade.	
Main flow	1. Registrar selects the enrolment, enters new grade and a reason (>= 10 chars). 2. System stores a PENDING request with the old grade.	
Alternative / exception flows	- No reason: rejected. - Section still OPEN: use normal lecturer entry instead.	
Postcondition	PENDING request exists.	

UC-R5 Approve / reject a grade change	Actor: Registrar (approver, different person)	FR: FR06
Precondition	PENDING request created by another registrar.	
Main flow	1. Approver reviews the request. 2. On approval ARS_API applies the grade and writes GRADE_AUDIT (POST_CLOSE) in one transaction; request becomes APPROVED. 3. On rejection the request becomes REJECTED; no grade change.	
Alternative / exception flows	- Approver is the requester: refused (ck_sod) and logged. - Lecturer or student calls the endpoint: 403.	
Postcondition	Audited, approved change (T9).	

5.5 Security Auditor

UC-A1 Review grade-change log	Actor: Security Auditor	FR: FR07
Precondition	Logged in as AUDITOR.	
Main flow	1. Auditor opens 'Grade audit'. 2. Backend returns GRADE_AUDIT rows (actor, old/new, time, record, reason) with filters.	
Alternative / exception flows	- Auditor tries to edit a grade (T6): refused; no edit control exists in the API.	
Postcondition	Log displayed, read-only.	

UC-A2 Review security events	Actor: Security Auditor	FR: FR07, FR08
Precondition	Logged in as AUDITOR.	
Main flow	1. Auditor opens 'Security events'. 2. Backend returns failed logins, lockouts and denied-access events with filters by date, actor, type.	
Alternative / exception flows	- Other roles requesting this endpoint (T5): 403 and ACCESS_DENIED event.	
Postcondition	Events displayed.	

UC-A3 View summary report	Actor: Security Auditor	FR: FR07
Precondition	Logged in as AUDITOR.	
Main flow	1. Auditor opens 'Reports'. 2. Backend returns aggregate counts (failed logins per day, grade changes per actor, post-closure changes).	
Alternative / exception flows	- Report must not contain national ID, DOB, phone or any student name.	
Postcondition	Aggregate report shown.	

6. Permission summary (preview of the M3 access matrix)

C = create, R = read, U = update. No role may delete academic records that have dependants, and nobody has update or delete rights on audit records.

Data / operation	Student	Lecturer	Registrar	Auditor
Own profile (STUDENTS)	R (own)	-	R	-
Student records incl. national_id, dob, phone	-	-	C R U	-
Courses / sections (COURSES, COURSE_OFFERINGS)	R (own, public part)	R (own sections)	C R U	-
Enrolments	R (own)	R (own sections)	C R U	-
Grade value - PUBLISHED sections	R (own)	R (own sections)	R	-
Grade value - OPEN section, direct entry	-	C U (own section)	-	-
Submit section (OPEN -> SUBMITTED)	-	Yes (own section)	-	-
Publish section (SUBMITTED -> PUBLISHED)	-	-	Yes	-
Grade change after closing (request)	-	-	Yes	-
Grade change after closing (approve/reject)	-	-	Yes (not own request)	-
GRADE_AUDIT	-	-	-	R
SECURITY_EVENTS	-	-	-	R
Delete / update audit records	No	No	No	No

7. Security requirements applied to SecARS

ID	Topic	How SecARS meets it	Milestone
SR01	Authentication	BCrypt/Argon2, lockout after 5 failures, HttpOnly/SameSite session, CSRF on, CORS limited to http://localhost:5173	M2 (login) / M3 (proof)
SR02	Authorization	@PreAuthorize per role on every endpoint; IDOR test; role only from server session	M3
SR03	DB least privilege	ARS_OWNER owns objects; APP_USER = CREATE SESSION + EXECUTE ARS_API; no SYS/SYSTEM in app	M3
SR04	Row-level security	VPD on ENROLLMENTS/grades driven by ARS_CTX; trust limits of the context documented	M3
SR05	SQL injection	Vulnerable search in ARS_LAB only; demo in-band + blind; fix with bind variables; re-test	M3
SR06	Secure coding	No string-built SQL; ORDER BY allow-list; validation; generic errors	M2 / M3
SR07	Database auditing	GRADE_AUDIT trigger/package; Unified Auditing; FGA on STUDENTS.national_id	M3
SR08	Separation of duties	Registrar cannot touch audit; auditor read-only; lecturer cannot approve; requester != approver	M3
SR09	Data protection	TCPS/TLS for app-to-DB (or TDE), tested; limits explained	M4
SR10	Secrets	Environment variables + .env.example only; wallets/keys outside Git	M2-M4
SR11	Backup and restore	Data Pump export, import to test schema, compare counts and hashes	M4
SR12	Masking	Mask names, national IDs, phones in restored copy only; keys/constraints intact	M4
SR13	Environment hardening	Listener on localhost/lab net; least-privilege service; before/after checklist	M3 / M4
SR14	Testing and logging	Unified test log: expected vs actual, evidence, date, tester	M3 / M4

8. Traceability FR - interface - data - tests

FR	Main tables	Interface	Tests / evidence
FR01	APP_USERS, SECURITY_EVENTS	POST /api/auth/login, POST /api/auth/logout, GET /api/me	SR01 login/lockout tests (M3 log)

FR	Main tables	Interface	Tests / evidence
FR02	STUDENTS, ENROLLMENTS, COURSE_OFFERINGS	GET /api/students/me/profile, /enrollments, /grades	T1, T2
FR03	COURSE_OFFERINGS, ENROLLMENTS	GET /api/lecturer/offerings, /offerings/{id}/students	T4
FR04	ENROLLMENTS, GRADE_AUDIT	POST /api/lecturer/offerings/{id}/grades, POST /api/lecturer/offerings/{id}/submit	T4, T9
FR05	STUDENTS, COURSES, COURSE_OFFERINGS, ENROLLMENTS	/api/registrar/students, /courses, /offerings, /enrollments, POST /api/registrar/offerings/{id}/publish	T3 (no direct table access); constraint tests
FR06	GRADE_CHANGE_REQUESTS, GRADE_AUDIT, ENROLLMENTS	POST /api/registrar/grade-changes, POST /api/registrar/grade-changes/{id}/approve | /reject	T9; SoD test
FR07	GRADE_AUDIT, SECURITY_EVENTS	GET /api/auditor/events, /grade-audit, /reports/summary	T5, T6
FR08	all read paths	query parameters on the GET endpoints above	T7, T8
FR09	(none)	global exception handler (all endpoints)	FR09 error-leak test (M3 log)
FR10	db/*.sql	db/01_admin_setup.sql ... db/security/*.sql, README.md	Clean-machine rebuild (M4)
FR11	(none)	React routes /student, /lecturer, /registrar, /auditor	Role-navigation check; T2/T4 (API refusal)
FR12	ARS_LAB (separate)	lab/isolated_SQLi_demo/, schema ARS_LAB	T7, T8, T13